Politecnico di Torino

Master of Science in Computer Engineering

Department of Control and Computer Engineering (DAUIN)

Database and Data Mining Group (DBDMG)

Large Language Models for Software Engineering

Volume II: Hands-on Laboratory Practicum

PyTorch foundations, Hugging Face architectures, PEFT, RAG, and SE agents

Complete laboratory implementation guide with full solutions

Step-by-step code walkthroughs, mathematical autograd derivations, and verification

Course Instructors:
Prof. Flavio Giobergia
Prof. Riccardo Coppola

Laboratory Curators:
Claudio Savelli
Tommaso Fulcini

Politecnico di Torino

Academic Year:
2026 / 2027

Target Program:
M.Sc. Computer Engineering

Course Credits:
6 CFU

Typeset with LaTeX, `tcolorbox`, and PGF/TikZ
Torino, Italy — A.Y. 2026/2027

Contents

I Foundations of deep learning and transformers **1**

1 Laboratory 01: Deep learning foundations with PyTorch **2**

1.1 Laboratory overview and macro perspective in LLM systems 2
1.2 PyTorch tensors, memory storage, and precision benchmarking 4
1.3 Computational graphs, autograd, and sensitivity mechanics 6
1.4 Building neural models, datasets, and the canonical training loop 9
1.5 Computer vision pipeline and data augmentation (CIFAR-10) 13
1.6 Multi-layer perceptron for classification (`SimpleNN`) 15
1.7 Transfer learning and parameter freezing with ResNet-18 17
1.8 Exercise 3 in-depth: analytical derivation and autograd DAG trace 20
1.9 Laboratory summary and engineering checklist 21

II Large language models for software engineering **24**

Part I

Foundations of deep learning and transformers

CHAPTER 1

Laboratory 01: Deep learning foundations with PyTorch

1.1 Laboratory overview and macro perspective in LLM systems

Before building modern foundation models with tens of billions of parameters, an AI software engineer must master the computational primitives that underpin all deep learning. Every Large Language Model (LLM)—from decoder-only transformers like LLaMA and GPT-4 to encoder-decoder models like T5—is executed on top of tensor arithmetic, automatic differentiation engines, and hardware-accelerated memory hierarchies.

1.1.1 Where this laboratory fits in the LLM lifecycle

To understand why we begin with PyTorch tensors, autograd, and transfer learning, consider the macroscopic lifecycle of an LLM shown in Figure 1.1.

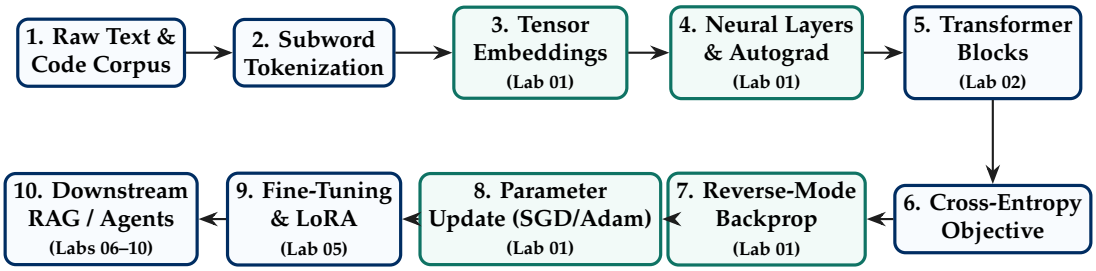


Figure 1.1: The end-to-end lifecycle of Large Language Models. Laboratory 01 establishes the atomic computational substrate: multidimensional tensors (Step 3), modular neural layers and forward graphs (Step 4), reverse-mode autograd (Step 7), and iterative optimization loops (Step 8).

Regardless of whether a model processes computer vision pixels ($32 \times 32 \times 3$) or textual tokens ($B \times T \times d_{\text{model}}$), the underlying execution pattern is strictly invariant across machine learning domains:

1. Input data is projected into multidimensional floating-point **tensors**.
2. A parameterized computational model (`nn.Module`) applies affine transformations interspersed with non-linear activations.
3. A scalar **loss function** evaluates prediction discrepancies against target ground truth.
4. An **automatic differentiation engine** traverses the Directed Acyclic Graph (DAG) in reverse, computing exact partial derivatives $\nabla_\theta \mathcal{L}$.
5. An **optimizer** updates parameters along the negative gradient surface.

1.1.2 Curriculum structure and pedagogical roadmap

As emphasized during the introductory laboratory lecture, the laboratory practicum adopts a progressive, modular structure:

- **Laboratory 01 (Foundational substrate)**: A self-contained, single-part introductory practicum establishing tensor manipulation, autograd mechanics, loss formulations, training loops, and transfer learning fundamentals.
- **Laboratories 02–10 (Dual-part advanced practicums)**: Starting from Laboratory 02, every subsequent session is partitioned into two complementary modules:
  1. *Architectural implementation from scratch* (e.g., implementing multi-head self-attention, positional encodings, subword tokenization, or LoRA adapters).
  2. *High-level industrial frameworks* (e.g., Hugging Face libraries such as `transformers`, `peft`, `trl`, and `accelerate`).

> 🎓 **Exam insights: Exam syllabus scope: NLP vs computer vision in LLM4SE**
>
> During the live laboratory session, the instructors explicitly clarified the examination boundaries: *students will not be evaluated on computer vision details, image data augmentation primitives (`RandomCrop`, `RandomHorizontalFlip`), or convolutional feature maps in the final exam.*
> Computer vision datasets (specifically CIFAR-10) are utilized exclusively in this first laboratory as an accessible, visually intuitive sandbox for high-dimensional tensors ($3 \times 32 \times 32$), batch slicing, multi-channel normalization, and pretrained transfer learning before students transition to discrete subword tokens, vocabulary embeddings, and causal attention masks.

1.1.3 Laboratory learning outcomes

Upon completing this laboratory, you will be able to:

- Manipulate PyTorch tensors, analyze underlying memory layouts (strides and contiguity), and evaluate hardware precision trade-offs (FP64 vs. FP32 vs. BF16).
- Trace and visualize Directed Acyclic Computational Graphs (DAGs) using reverse-mode automatic differentiation.
- Subclass `torch.nn.Module`, implement custom parameter layers, and configure optimizers and loss functions.
- Author robust, multi-epoch training loops from scratch with metric tracking and loss visualization.
- Compute per-channel dataset statistics (mean and standard deviation) and build data transformation and augmentation pipelines with `torchvision.transforms`.
- Implement multi-layer perceptron classifiers (`SimpleNN`) and construct evaluation loops with `model.eval()` and `torch.no_grad()`.
- Apply transfer learning with pretrained deep residual networks (ResNet-18), freeze convolutional feature extractors, and adapt classification heads for new downstream domains.

1.2 PyTorch tensors, memory storage, and precision benchmarking

In deep learning, a **tensor** is a generalized geometric object comprising a multidimensional array of numbers characterized by a specific data type (`dtype`) and memory device (`device`).

1.2.1 Importing the environment and OpenMP configuration

To ensure robust multi-threaded tensor execution across Apple Silicon and Linux environments, we initialize the required libraries and set environment flags:

```
# Import necessary libraries
import os
os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset
import matplotlib.pyplot as plt

import torchvision
import torchvision.transforms as transforms
```

Listing 1.1: Initial library imports and environment configuration (Cell 2).

1.2.2 Basic tensor algebra

A PyTorch tensor acts as a multidimensional view over a contiguous chunk of flat physical memory:

```
# Create a tensor
a = torch.tensor([1.0, 2.0, 3.0])

# Perform basic tensor operations
b = torch.tensor([4.0, 5.0, 6.0])
c = a + b  # Element-wise addition
print(f"Tensor c: {c}")

# Check tensor's shape and type
print(f"Shape of tensor c: {c.shape}")
print(f"Data type of tensor c: {c.dtype}")
```

Listing 1.2: Tensor creation, element-wise addition, and inspection (Cell 4).

Executing Listing 1.2 outputs:

```
Tensor c: tensor([5., 7., 9.])
Shape of tensor c: torch.Size([3])
Data type of tensor c: torch.float32
```

1.2.3 Floating-point representations and empirical speed benchmarking

A critical engineering decision in modern AI is the choice of numerical precision. Standard scientific computing defaults to double precision (`torch.float64` or FP64), requiring 64 bits per scalar (1 sign bit, 11 exponent bits, 52 mantissa bits). In contrast, deep learning predominantly

operates with single precision (`torch.float32` or FP32: 1 sign, 8 exponent, 23 mantissa bits) or half precision (FP16 / BF16).

To observe the substantial performance differential directly, the laboratory benchmarks matrix multiplication ($\mathbf{M}_1 \times \mathbf{M}_2$) on two 1000×1000 matrices across 100 repetitions:

```
from timeit import timeit

mat_size = 1000

M1_64 = torch.randn(mat_size, mat_size, dtype=torch.float64)
M2_64 = torch.randn(mat_size, mat_size, dtype=torch.float64)

M1_32 = torch.randn(mat_size, mat_size)
M2_32 = torch.randn(mat_size, mat_size)

t64 = timeit(lambda: M1_64 @ M2_64, number=100)
t32 = timeit(lambda: M1_32 @ M2_32, number=100)

print(f"Time for matrix multiplication (float64): {t64:.4f}s")
print(f"Time for matrix multiplication (float32): {t32:.4f}s")
```

Listing 1.3: Empirical matrix multiplication benchmark comparing FP64 and FP32 (Cell 6).

Precision Format	Bit Width	Dynamic Range	Benchmark	Relative Throughput
`torch.float64` (FP64)	64 bits	$\approx 10^{\pm 308}$	0.6864 s	1.0× (Baseline)
`torch.float32` (FP32)	32 bits	$\approx 10^{\pm 38}$	0.1877 s	**3.65**× faster
`torch.float16` (FP16)	16 bits	$\approx 10^{\pm 5}$	Fast on TC	Up to 8× vs FP32
`torch.bfloat16` (BF16)	16 bits	$\approx 10^{\pm 38}$	Modern GPUs	Stable for LLMs

Table 1.1: Numerical precision comparison in deep learning. Transitioning from FP64 to FP32 yields an immediate $\approx 3.7\times$ speedup due to vector SIMD register capacity and halved memory bandwidth demands.

1.2.4 Hardware acceleration, device dispatch, and the overloaded `.to()` method

A primary advantage of PyTorch over NumPy is native support for hardware accelerators (NVIDIA CUDA GPUs and Apple Silicon MPS cores). To write hardware-agnostic code, deep learning engineers dynamically discover the execution target:

```
# Determine the optimal acceleration backend
if torch.cuda.is_available():
    device = torch.device("cuda")
elif torch.backends.mps.is_available():
    device = torch.device("mps")
else:
    device = torch.device("cpu")

print(f"Active execution device: {device}")
```

Listing 1.4: Hardware-agnostic device configuration and tensor dispatch.

During the laboratory lecture, the instructor emphasized the dual, polymorphic role of the `.to()` method in PyTorch:

1. **Type Casting**: Transforming numeric representations without altering physical hardware location (e.g., `x.to(torch.float16)` or `x.double()`).

2. **Hardware Placement**: Moving data and model parameters across memory boundaries (e.g., from host CPU RAM to GPU High-Bandwidth Memory via `x.to(device)`).

Exam insights: The strict homogeneous device invariant

All operands in any tensor computation (the model parameters **W**, **b**, input features **x**, and target labels **y**) **must strictly reside on the exact same physical device**.
If an input tensor on CPU is passed into a neural network situated in GPU VRAM, PyTorch immediately halts execution with a fatal runtime exception:

```
RuntimeError: Expected all tensors to be on the same device, but found at
    least two devices, cuda:0 and cpu!
```

Therefore, every robust training and inference pipeline must explicitly ensure consistent device placement before initiating the forward pass.

External deep dive: Why deep learning and LLMs mandate lower numerical precision

Standard scientific computing defaults to double precision (`torch.float64` or FP64), requiring 64 bits per scalar (1 sign bit, 11 exponent bits, 52 mantissa bits). In contrast, deep learning and natural language processing predominantly operate with single precision (`torch.float32` or FP32), half precision (`torch.float16` / `torch.bfloat16`), or quantized formats (8-bit and 4-bit).
As highlighted by the laboratory instructors, two primary engineering rationales govern this preference:

1. **Computational Throughput**: Modern GPU Tensor Cores and vector SIMD arithmetic units execute 16-bit matrix multiplications orders of magnitude faster than 64-bit doubles.

2. **GPU VRAM Capacity Bottleneck**: High-bandwidth memory (HBM) on AI accelerators is severely limited and prohibitively expensive. Every model parameter in FP32 requires 4 bytes of memory. Halving precision to FP16 or BF16 (2 bytes per parameter) instantaneously **doubles the number of parameters that fit within the same physical VRAM** with virtually zero degradation in task performance.

This capacity bottleneck provides the core motivation for model quantization, which compresses weights down to 8 bits (1 byte) or 4 bits (0.5 bytes) for efficient production deployment.

1.3 Computational graphs, autograd, and sensitivity mechanics

In classical machine learning, engineers derived analytic gradient formulas by hand and coded them manually. In modern deep learning, the framework dynamically constructs a **Directed Acyclic Graph (DAG)** of operations as forward computations execute (PyTorch's *Define-by-Run* paradigm).

1.3.1 Autograd mechanics, leaf nodes, and `requires_grad`

When a tensor is declared with `requires_grad=True`, PyTorch tracks every operation applied to it:

- **Leaf Tensors**: Tensors explicitly created by the user (e.g., weight matrices $\mathbf{W}$ and biases $\mathbf{b}$). Their gradients are accumulated in the `.grad` attribute.
- **Non-Leaf Tensors**: Intermediate results produced by operator nodes (e.g., activations $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$). Their gradients are computed dynamically during backpropagation but discarded after use to conserve memory unless `.retain_grad()` is invoked.
- **`grad_fn`**: Every non-leaf tensor stores a reference to the backward function node that created it (e.g., `AddBackward0`, `MulBackward0`).

Exam insights: `requires_grad` in training vs inference

When `requires_grad=True` is set, PyTorch allocates dedicated memory buffers for gradient tracking and builds the dynamic DAG during the forward pass.
While essential during model training, retaining `requires_grad=True` during **inference** incurs substantial, unnecessary overhead:

- It wastes GPU VRAM by storing intermediate activation histories that will never be used for backward differentiation.
- It introduces computational latency by constructing graph metadata.

Consequently, during inference or validation, engineers must always de-activate autograd graph construction using `with torch.no_grad():` or by setting `param.requires_grad = False`.

1.3.2 External authority: Andrej Karpathy's Micrograd engine

To demystify how PyTorch implements reverse-mode automatic differentiation without treating it as a black box, Andrej Karpathy developed *Micrograd*—a scalar-level autograd engine that mirrors PyTorch's C++ autograd internals in pure Python.

External deep dive: Andrej Karpathy's Micrograd: Autograd stripped to its core

In Micrograd, every number is wrapped in a lightweight `Value` object. Each object maintains:

- `data`: The forward numerical scalar.
- `grad`: The accumulated adjoint partial derivative $\partial \mathcal{L}/\partial \text{self}$.
- `_prev`: Pointers to upstream parent nodes in the computation graph.
- `_backward`: A local closure encapsulating the single-step multivariate chain rule.

```
class Value:
    def __init__(self, data, _children=(), _op=''):
        self.data = data
        self.grad = 0.0
        self._backward = lambda: None
        self._prev = set(_children)
        self._op = _op

    def __add__(self, other):
```

```
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data + other.data, (self, other), '+')
        def _backward():
            self.grad += 1.0 * out.grad
            other.grad += 1.0 * out.grad
        out._backward = _backward
        return out

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data * other.data, (self, other), '*')
        def _backward():
            self.grad += other.data * out.grad
            other.grad += self.data * out.grad
        out._backward = _backward
        return out

    def backward(self):
        # Topological sort of all nodes in the DAG
        topo = []
        visited = set()
        def build_topo(v):
            if v not in visited:
                visited.add(v)
                for child in v._prev:
                    build_topo(child)
                topo.append(v)
        build_topo(self)

        # Base gradient: dLoss / dLoss = 1.0
        self.grad = 1.0
        # Traverse graph in reverse topological order
        for node in reversed(topo):
            node._backward()
```

Listing 1.5: Core structure of Andrej Karpathy's scalar autograd engine.

When calling `loss.backward()`, the engine conducts a **depth-first topological sort** of the DAG. It initializes $\frac{\partial \mathcal{L}}{\partial \mathcal{L}} = 1.0$ and propagates scalar derivatives backward through every closure. PyTorch operates on the exact same mathematical principle, scaling scalar closures to multidimensional tensor Jacobian-vector products.

Intuition & analogy: 3Blue1Brown's sensitivity view of backpropagation

Grant Sanderson (3Blue1Brown) articulates backpropagation not as abstract multivariable calculus, but as **sensitivity propagation**. Imagine giving parameter θ_1 a slight physical nudge of size ϵ. That nudge ripples forward through the graph:

$$\theta_1 \to \theta_1 + \epsilon \implies a = (\theta_1 + \epsilon)\theta_2 = a + \epsilon\theta_2. \tag{1.1}$$

The change in a wiggles the downstream product b, which shifts difference c, which finally scales loss $\mathcal{L}$ by an amount proportional to ϵ. The partial derivative $\frac{\partial \mathcal{L}}{\partial \theta_1}$ is simply the ratio of the final output wiggle to the initial parameter nudge as $\epsilon \to 0$. Reverse-mode automatic differentiation systematically traverses these interconnected sensitivity levers

backward from output to input.

1.4 Building neural models, datasets, and the canonical training loop

The laboratory transitions from primitive tensor operations to structured neural modeling.

1.4.1 Defining models with `torch.nn.Module`

In PyTorch, all neural layers and complete architectures inherit from `nn.Module`:

```
# Define a simple linear model
class SimpleLinearModel(nn.Module):
    def __init__(self, input_size, output_size):
        super(SimpleLinearModel, self).__init__()
        self.linear = nn.Linear(input_size, output_size)

    def forward(self, x):
        return self.linear(x)

# Initialize the model and configure device
model = SimpleLinearModel(1, 1)
device = torch.device("cuda" if torch.cuda.is_available() else ("mps" if
    torch.backends.mps.is_available() else "cpu"))
model = model.to(device)

print(f"Device: {device}")
print("Initial Weight:", model.linear.weight)
print("Initial Bias:", model.linear.bias)
```

Listing 1.6: Defining a single-input single-output linear model (Cells 11–18).

Intuition & analogy: Interactive classroom quiz: Parameter counting in linear layers

During the live laboratory walkthrough, the instructor posed an interactive puzzle testing students' structural understanding of linear projections:

1. **Case A ($d_{\text{in}} = 5, d_{\text{out}} = 1$)**: How many weights and biases are instantiated?
   - *Solution*: Exactly **5 weights** and **1 bias** (total = 6 parameters). Each input scalar is scaled by an individual weight, summed together, and shifted by a single scalar bias: $\hat{y} = \sum_{j=1}^{5} w_j x_j + b$.
2. **Case B ($d_{\text{in}} = 1, d_{\text{out}} = 5$)**: How many weights and biases are instantiated?
   - *Solution*: Exactly **5 weights** and **5 biases** (total = 10 parameters!).
   - *Geometric intuition*: As the instructor emphasized, moving from 1 to 5 outputs is conceptually equivalent to *parallelizing the data across 5 independent linear hyperplanes*. Each output feature neuron requires its own private scaling factor and its own translation threshold: $\hat{y}_k = w_k x + b_k$ for $k \in \{1, \dots, 5\}$.

In general, a fully connected linear layer mapping from d_{in} to d_{out} computes:

$$\mathbf{y} = \mathbf{x}\mathbf{W}^\top + \mathbf{b}, \quad \text{where } \mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}},\ \mathbf{b} \in \mathbb{R}^{d_{\text{out}}}, \tag{1.2}$$

instantiating exactly $P = d_{\text{out}} \cdot d_{\text{in}} + d_{\text{out}}$ trainable parameters.

1.4.2 Configuring criterion and optimizer

Every learning pipeline couples an objective metric with a gradient-directed stepping policy:

- **Criterion (Loss Function)**: Evaluates the magnitude of prediction error against ground-truth targets. The instructor framed the loss function as an *informed oracle*: because the loss function observes the true target y, it calculates the exact scalar error $\mathcal{L}(\hat{y}, y)$ required to steer the parameters.
  - *Continuous Regression*: Mean Squared Error (MSE):

$$\mathcal{L}_{\text{MSE}}(\hat{y}, y) = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2. \tag{1.3}$$

  - *Discrete Classification*: Multi-class Cross-Entropy Loss:

$$\mathcal{L}_{\text{CE}}(\hat{\mathbf{y}}, y) = -\log\left(\frac{e^{\hat{y}_y}}{\sum_{j=1}^{C} e^{\hat{y}_j}}\right). \tag{1.4}$$

- **Optimizer**: Dictates how parameters move in response to the backward gradient $\nabla_\theta \mathcal{L}$. While the loss function provides the *direction* of descent, the optimizer governs the *step size and trajectory dynamics*.

Exam insights: LLMs as giant classification models

The instructor highlighted an essential theoretical link bridging primitive neural networks with foundation models: *Large Language Models are, at their computational core, massive multi-class classification networks*.
Instead of classifying an image into 10 classes, an autoregressive LLM projects hidden representation $\mathbf{h}_t \in \mathbb{R}^{d_{\text{model}}}$ through an unembedding projection layer $\mathbf{W}_U \in \mathbb{R}^{V \times d_{\text{model}}}$ to output logits over a vocabulary V containing 32,000 to 128,000+ tokens, supervised by the exact same Cross-Entropy Loss formulation.

External deep dive: Optimizer dynamics: From SGD to Adam's oscillation damping

In pure Gradient Descent, updates follow:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t). \tag{1.5}$$

However, modern deep learning mandates two critical modifications:

1. **Stochastic Mini-Batching**: Evaluating the true gradient over millions of data points simultaneously is computationally impossible and exceeds GPU memory. Mini-batches (e.g., $B = 64$ or 256) provide stochastic estimates of the gradient, introducing beneficial noise that helps parameters escape shallow saddle points.
2. **Adaptive Moment Estimation (Adam)**: The instructor presented an intuitive physical view of Adam's stability. If a parameter's gradient oscillates wildly between consecutive batches (e.g., pointing positive in step t, negative in step $t+1$, and positive in step $t+2$), standard SGD wastes energy zigzagging across valley walls. Adam tracks exponentially decaying moving averages of past gradients (m_t)

and squared gradients (v_t):

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2. \tag{1.6}$$

When high-frequency oscillation occurs, v_t grows relative to m_t, and the denominator $\sqrt{\hat{v}_t} + \epsilon$ automatically **dampens the effective step size**, stabilizing training without manual learning rate decay.

```
criterion = nn.MSELoss()
optimizer = optim.SGD(model.parameters(), lr=0.01)
```

Listing 1.7: Configuring loss function and SGD optimizer (Cell 20).

1.4.3 Exercise 1: Synthetic regression dataset

The laboratory tasks the student with generating a synthetic dataset of $N = 2048$ points drawn from a known underlying linear relation $y = w_{\text{true}} \cdot x + b_{\text{true}} + \epsilon$, batched using `TensorDataset` and `DataLoader`.

```
# Create a dataset (synthetic linear regression dataset)
n_pts = 2048

# Ground-truth parameters to be recovered by the model
w_true = 2.0
b_true = 1.0

# Generate random inputs X ~ N(0, 1) of shape (n_pts, 1)
X = torch.randn(n_pts, 1)

# Generate targets y with additive Gaussian noise
y = w_true * X + b_true + 0.1 * torch.randn(n_pts, 1)

# Package into DataLoader
dataset = TensorDataset(X, y)
trainloader = DataLoader(dataset, batch_size=256, shuffle=True)
```

Listing 1.8: Verified solution for synthetic dataset generation (Cell 22).

External deep dive: Dataset abstractions and the mechanics of steps vs epochs

During the laboratory lecture, the instructor meticulously unpacked the roles of `TensorDataset` and `DataLoader`:

- **`TensorDataset`**: Serves as a contiguous indexing wrapper binding feature tensors $\mathbf{X} \in \mathbb{R}^{N \times 1}$ and label tensors $\mathbf{y} \in \mathbb{R}^{N \times 1}$. Slicing `dataset[i]` yields tuple $(\mathbf{x}_i, y_i)$.
- **`DataLoader`**: Manages multi-threaded mini-batch generation, memory pinning, and sample permutation:
  1. **Batch Slicing**: Subdivides the dataset into mini-batches of size $B = 256$.
  2. **Shuffling (`shuffle=True`)**: Permutes the row indices at the start of every epoch. This ensures that the model does not encounter sample pairings in identical sequence, preventing cyclical limit cycles in parameter space.

Steps vs. Epochs: A fundamental metric distinction emphasized in the course:

- An **Epoch** corresponds to one complete traversal over the entire training corpus ($N = 2048$ samples).
- A **Step** (or iteration) corresponds to a single parameter update evaluated on one mini-batch ($B = 256$).

The number of steps per epoch is given by:

$$S_{\text{epoch}} = \left\lceil \frac{N}{B} \right\rceil = \frac{2048}{256} = 8 \text{ steps per epoch.} \tag{1.7}$$

Over $E = 50$ epochs, the model performs exactly $S_{\text{total}} = 50 \times 8 = 400$ parameter update steps.

1.4.4 Exercise 1: Multi-epoch training loop with parameter tracking

To observe convergence, we implement the **5 canonical steps** of the PyTorch training loop:

1. **Gradient Reset**: `optimizer.zero_grad()` erases residual gradient accumulations from preceding mini-batches.
2. **Forward Evaluation**: `outputs = model(inputs)` traverses the neural network's forward DAG.
3. **Error Quantification**: `loss = criterion(outputs, labels)` evaluates scalar discrepancy.
4. **Adjoint Backpropagation**: `loss.backward()` traverses the DAG in reverse via autograd.
5. **Parameter Step**: `optimizer.step()` translates weights along the gradient trajectory.

```
# Training loop
num_epochs = 50

losses = []
weights = []
biases = []

model.train()
for epoch in range(num_epochs):
    running_loss = 0.0
    for inputs, labels in trainloader:
        inputs, labels = inputs.to(device), labels.to(device)

        # 1. Zero the parameter gradients from previous step
        optimizer.zero_grad()

        # 2. Forward pass: compute predictions
        outputs = model(inputs)

        # 3. Compute scalar loss
        loss = criterion(outputs, labels)

        # 4. Backward pass: compute gradients via autograd
```

```
        loss.backward()

        # 5. Optimizer step: update weights along negative gradient
        optimizer.step()

        # Track metrics at each step
        losses.append(loss.item())
        weights.append(model.linear.weight.item())
        biases.append(model.linear.bias.item())

        running_loss += loss.item()

    if (epoch + 1) % 10 == 0 or epoch == 0:
        print(f"Epoch [{epoch+1}/{num_epochs}], Loss:
            {running_loss/len(trainloader):.4f}")
```

Listing 1.9: Verified solution for the 50-epoch training loop with metric tracking (Cell 24).

```
Epoch [1/50], Loss: 2.2321
Epoch [10/50], Loss: 0.1246
Epoch [20/50], Loss: 0.0140
Epoch [30/50], Loss: 0.0099
Epoch [40/50], Loss: 0.0097
Epoch [50/50], Loss: 0.0097
Learned weights: tensor([[2.0000]])
Learned biases: tensor([0.9988])
```

The model successfully converges from arbitrary random initialization to recover the true underlying slope $w \to 2.0000$ and intercept $b \to 0.9988$, matching the 3-panel visualization in Figure 1.2.

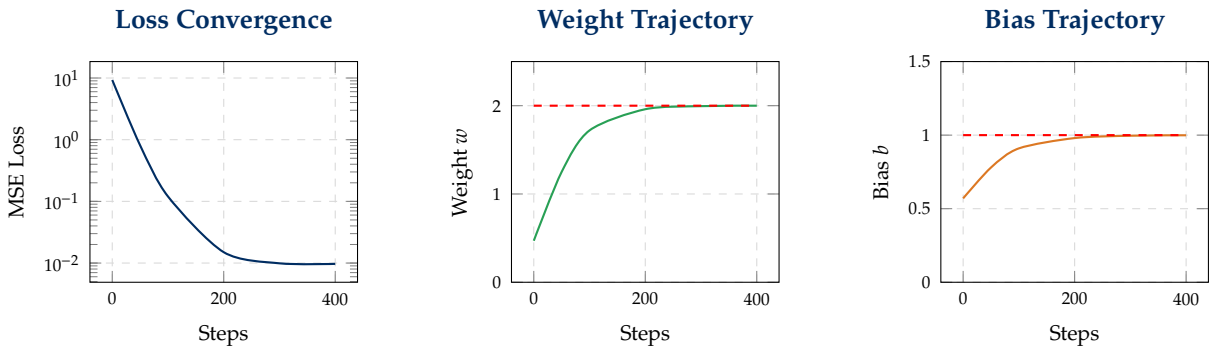


Figure 1.2: Visualizing optimization dynamics across 50 epochs (Cell 25). The mean squared error decays logarithmically towards the irreducible noise floor ($\sigma^2 = 0.01$), while weight w asymptotically reaches 2.0 and bias b reaches 1.0.

1.5 Computer vision pipeline and data augmentation (CIFAR-10)

Moving beyond 1D synthetic scalars, Section 2 of the laboratory introduces the **CIFAR-10** benchmark (60,000 32×32 color images across 10 classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck).

1.5.1 Channel ordering: PyTorch vs Matplotlib

PyTorch standardizes on the **Channel-First** format $[B, C, H, W]$, where $C = 3$ color channels (Red, Green, Blue) precede height $H = 32$ and width $W = 32$. However, Matplotlib and visualization libraries expect the **Channel-Last** format $[H, W, C]$. To render a PyTorch image tensor with `plt.imshow()`, one must invoke:

$$\mathbf{x}_{\text{plot}} = \mathbf{x}_{\text{torch}}.\text{permute}(1, 2, 0). \tag{1.8}$$

1.5.2 Exercise 2: Computing dataset mean and standard deviation from scratch

Standardization accelerates gradient descent by ensuring zero-centered inputs with unit variance across all feature channels:

$$\tilde{x}_{c,i,j} = \frac{x_{c,i,j} - \mu_c}{\sigma_c} \quad \text{for } c \in \{R, G, B\}. \tag{1.9}$$

In Cell 33, students are tasked with computing μ_c and σ_c across all 50,000 training images scaled to $[0, 1]$:

```
# Download initial dataset to compute raw statistics
import os
dataset_dir = os.path.expanduser('~/data')

trainset = torchvision.datasets.CIFAR10(
    root=dataset_dir, train=True, download=True, transform=transforms.ToTensor()
)

# trainset.data is a numpy array of shape (50000, 32, 32, 3) with uint8 values [0,
    255]
# Scale to [0, 1] floating-point range
scaled_data = trainset.data / 255.0

# Compute mean and standard deviation across image batches, heights, and widths (axes
    0, 1, 2)
images_mean = tuple(scaled_data.mean(axis=(0, 1, 2)).tolist())
images_std = tuple(scaled_data.std(axis=(0, 1, 2)).tolist())

print(f"Calculated Mean (R, G, B): {images_mean}")
print(f"Calculated Std  (R, G, B): {images_std}")
```

Listing 1.10: Verified solution for calculating dataset mean and standard deviation per channel (Cell 33).

The resulting values match the canonical CIFAR-10 population statistics:

$$\boldsymbol{\mu} = (0.4914, 0.4822, 0.4465), \quad \boldsymbol{\sigma} = (0.2470, 0.2435, 0.2616). \tag{1.10}$$

1.5.3 Data augmentation and visualization pipeline

Data augmentation synthetically expands the training distribution by applying label-preserving geometric perturbations. In Cell 37, the laboratory constructs a visualization pipeline applying `RandomHorizontalFlip` and `RandomCrop` with reflection padding:

```
viz_transform = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    # Reflect border pixels to handle crop margins
    transforms.RandomCrop(32, padding_mode='reflect', padding=5)
])
```

```

# Extract a sample image tensor and convert to PIL format
images, labels = next(iter(trainloader))
im = transforms.ToPILImage()(images[10])
n_transforms = 9

fig, ax = plt.subplots(3, 3, figsize=(4, 4))

# Slot 0: Original unaugmented image
ax[0, 0].imshow(im)
ax[0, 0].set_title('Original', fontsize=8)
ax[0, 0].axis('off')

# Slots 1-8: Stochastic variations generated by the transform
for i in range(1, n_transforms):
    r, c = divmod(i, 3)
    ax[r, c].imshow(viz_transform(im))
    ax[r, c].set_title(f'Aug {i}', fontsize=8)
    ax[r, c].axis('off')

plt.tight_layout()
```

Listing 1.11: Verified solution for data augmentation visualization (Cell 37).

1.6 Multi-layer perceptron for classification (`SimpleNN`)

In Section 2.0, the laboratory constructs a Multi-Layer Perceptron (MLP) for 10-class image classification.

1.6.1 Mathematical formulation of the architecture

The network maps a 3D image tensor into a flattened vector, passes it through two hidden layers with Rectified Linear Unit (ReLU) activations, and outputs unnormalized class logits:

$$\mathbf{x}_{\text{flat}} = \text{vec}(\mathbf{x}) \in \mathbb{R}^{32\times32\times3} = \mathbb{R}^{3072}, \tag{1.11}$$

$$\mathbf{h}_1 = \text{ReLU}(\mathbf{W}_1\mathbf{x}_{\text{flat}} + \mathbf{b}_1) \in \mathbb{R}^{512}, \qquad \mathbf{W}_1 \in \mathbb{R}^{512\times3072},\ \mathbf{b}_1 \in \mathbb{R}^{512}, \tag{1.12}$$

$$\mathbf{h}_2 = \text{ReLU}(\mathbf{W}_2\mathbf{h}_1 + \mathbf{b}_2) \in \mathbb{R}^{256}, \qquad \mathbf{W}_2 \in \mathbb{R}^{256\times512},\ \mathbf{b}_2 \in \mathbb{R}^{256}, \tag{1.13}$$

$$\hat{\mathbf{y}} = \mathbf{W}_3\mathbf{h}_2 + \mathbf{b}_3 \in \mathbb{R}^{10}, \qquad \mathbf{W}_3 \in \mathbb{R}^{10\times256},\ \mathbf{b}_3 \in \mathbb{R}^{10}. \tag{1.14}$$

Figure 1.3 illustrates the end-to-end architectural topology of `SimpleNN`, visualizing the tensor dimensions, fully connected weight matrices, non-linear ReLU activations, and reverse-mode gradient flow.

1.6.2 Exercise: Implementing `SimpleNN`

```
class SimpleNN(nn.Module):
    def __init__(self):
        super(SimpleNN, self).__init__()
        # Linear layer 1: 3072 input features -> 512 hidden features
        self.fc1 = nn.Linear(32 * 32 * 3, 512)
        # Linear layer 2: 512 hidden features -> 256 hidden features
        self.fc2 = nn.Linear(512, 256)
        # Output layer: 256 hidden features -> 10 class logits
        self.fc3 = nn.Linear(256, 10)
```

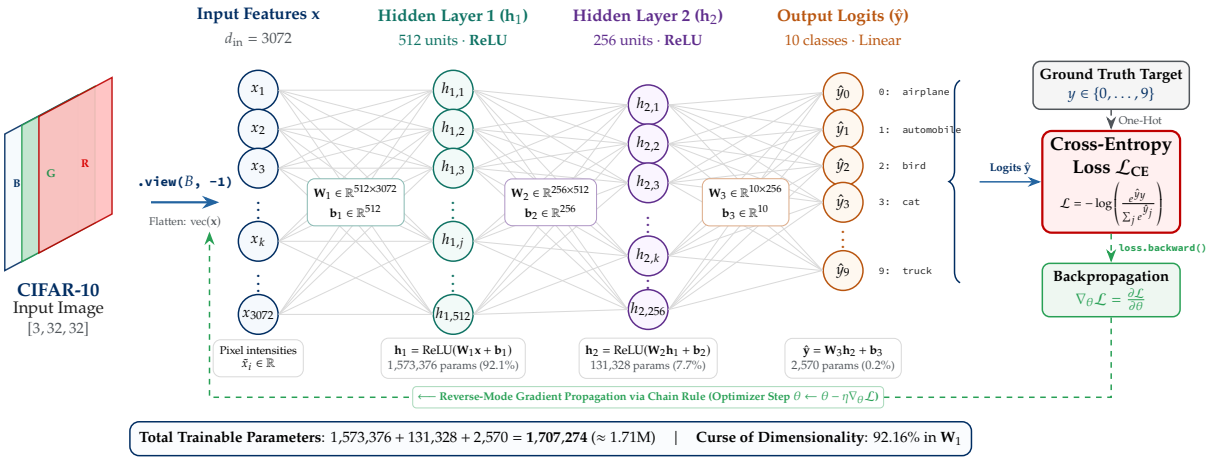


Figure 1.3: Architectural topology of the `SimpleNN` Multi-Layer Perceptron (MLP) for CIFAR-10 classification (Section 1.6.1). An input color image $[3 \times 32 \times 32]$ is flattened into feature vector $\mathbf{x} \in \mathbb{R}^{3072}$, sequentially propagated through two hidden fully-connected layers with non-linear ReLU activations (512 and 256 units), and projected into 10 unnormalized class logits $\hat{\mathbf{y}} \in \mathbb{R}^{10}$. Training is supervised by Cross-Entropy Loss with gradients propagated in reverse mode.

```
    def forward(self, x):
        # Flatten input: preserve batch dimension, collapse (C, H, W)
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        # Logits output (no activation needed; CrossEntropyLoss applies Softmax
            internally)
        x = self.fc3(x)
        return x
```

Listing 1.12: Verified implementation of `SimpleNN` architecture (Cell 42).

1.6.3 Evaluation function and gradient deactivation

Validation requires disabling parameter updates to prevent memory leaks and ensure deterministic evaluation:

```
def val_model(model, testloader):
    model.eval()  # Switch to evaluation mode (deactivates dropout/batchnorm training
        behavior)
    correct = 0
    total = 0
    with torch.no_grad():  # Disable autograd DAG construction to conserve memory
        for inputs, labels in testloader:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    accuracy = 100 * correct / total
    print(f"Accuracy of the network on test images: {accuracy:.2f}%")
```

Listing 1.13: Validation routine with `model.eval()` and `torch.no_grad()` (Cell 48).

Prior to training, random guessing on a 10-class dataset yields an accuracy of $\approx 10.0\%$. After 5 epochs of training via Listing 1.9, the multi-layer perceptron achieves approximately 47.5% accuracy.

1.7 Transfer learning and parameter freezing with ResNet-18

While training an MLP from scratch yields $\approx 47\%$ accuracy on CIFAR-10, modern engineering relies on **transfer learning**: reusing representations acquired by deep models pretrained on massive datasets (such as ImageNet's 1.2 million images).

1.7.1 Deep residual learning (ResNet)

Introduced in [1], Deep Residual Networks solve the vanishing gradient problem in networks with dozens of layers by introducing **identity shortcut connections**:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{\mathbf{W}_i\}) + \mathbf{x}. \tag{1.15}$$

During backpropagation, the additive residual connection ensures gradients flow unimpeded directly to early layers:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \left(\frac{\partial \mathcal{F}}{\partial \mathbf{x}} + \mathbf{I} \right). \tag{1.16}$$

Even if the learned Jacobian $\frac{\partial \mathcal{F}}{\partial \mathbf{x}}$ vanishes, the identity term $\mathbf{I}$ guarantees an uninterrupted gradient conduit.

1.7.2 Exercise: Adapting pretrained ResNet-18 for CIFAR-10

The base `models.resnet18` has an output fully connected layer (`fc`) sized for 1000 ImageNet classes. In Cell 65, the student replaces this head with a new 10-class linear projection. Figure 1.4 illustrates the transfer learning pipeline, contrasting the pre-trained ImageNet source model against the adapted CIFAR-10 downstream architecture.

```
from torchvision import models

# Load ResNet-18 with ImageNet weights
pretrained_model = models.resnet18(weights='DEFAULT')

# Access original input feature dimension (512) and replace head
pretrained_model.fc = nn.Linear(pretrained_model.fc.in_features, 10)

# Transfer model to active device
pretrained_model = pretrained_model.to(device)
```

Listing 1.14: Loading pretrained ResNet-18 and replacing the classifier head (Cell 65).

Fine-tuning this model for 5 epochs elevates validation accuracy from 47.5% to over **82.4**%.

1.7.3 Exercise: Transfer to CIFAR-100 and parameter freezing

In Section 2.3, the challenge shifts to fine-tuning the model for **CIFAR-100** (100 fine-grained classes). To prevent overwriting the valuable pretrained visual feature extractors and drastically accelerate training, we **freeze all backbone layers** and only train the newly attached classification head:

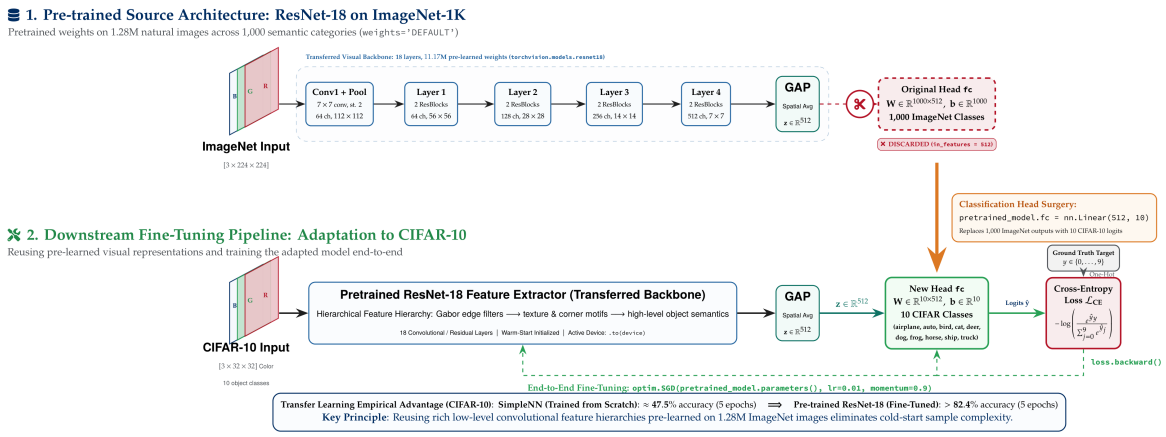


Figure 1.4: Architectural topology of transfer learning and fine-tuning with ResNet-18 on CIFAR-10 (Section 1.7.2). (Top) The source ResNet-18 model pre-trained on ImageNet-1K (1.28M images, 1,000 categories) features 18 residual convolutional layers and an original 1,000-class classification head. (Middle) Classification head surgery replaces `pretrained_model.fc` with a new linear projection $\mathbf{W} \in \mathbb{R}^{10\times512}$, $\mathbf{b} \in \mathbb{R}^{10}$. (Bottom) End-to-end downstream fine-tuning on CIFAR-10 ($[3 \times 32 \times 32]$), propagating gradients through both the new head and the pre-trained feature extractor, boosting classification accuracy from 47.5% (MLP from scratch) to > 82.4% in just 5 epochs.

```python
# Modify classification head for 100 classes
model = pretrained_model
model.fc = nn.Linear(model.fc.in_features, 100)
model = model.to(device)

# Step 1: Freeze all layers across the network
for param in model.parameters():
    param.requires_grad = False

# Step 2: Unfreeze ONLY the classification head parameters
for param in model.fc.parameters():
    param.requires_grad = True

# Optimizer tracks ONLY the head parameters
optimizer = optim.SGD(model.fc.parameters(), lr=0.01, momentum=0.9, weight_decay=5e-4)
```

Listing 1.15: Freezing backbone parameters and unfreezing the classification head (Cells 72–75).

Configuration	Total Parameters	Trainable Parameters	Trainable Ratio
Full ResNet-18 Fine-Tuning	11,176,512	11,176,512	100.0%
Frozen Backbone + Head Only	11,227,584	**51,300**	**0.457**%

Table 1.2: Parameter efficiency comparison under layer freezing. By training only the newly attached classification head (`model.fc`), the number of active gradients drops by over 99.5%, serving as a direct conceptual precursor to LoRA (Laboratory 05).

1.7.4 The foundation model paradigm: Bridging ResNet in vision and BERT in NLP

During the laboratory lecture, the instructor established an indispensable pedagogical bridge connecting convolutional vision backbones with large language models. In industrial machine learning, the era of training massive architectures from scratch on modest consumer hardware has drawn to a close:

- **The Economics of Pretraining**: Pretraining foundation models (such as BERT, RoBERTa, LLaMA, or deep vision backbones) demands millions of dollars in compute, warehouse-scale GPU clusters, and massive data ingestion pipelines. More than 99% of practitioners and industrial teams cannot pretrain foundation models from scratch.
- **Latent Space Representations**: Pretrained foundation models arrive equipped with non-random, semantically structured weights. In vision, ResNet maps raw pixels into rich hierarchical visual features (edges → textures → object contours). In NLP, an encoder like BERT maps raw token sequences into structured latent embeddings $\mathbf{z} \in \mathbb{R}^{d_{\text{model}}}$ capturing syntactic grammar, context, and semantic co-occurrence.

Strategy	Training From Scratch	Full Fine-Tuning	Linear Probing (Freezing)
Weight Initialization	Random Gaussian / Xavier	Pretrained foundation weights	Pretrained foundation weights
Active Gradients	All layers (100%)	All layers (100%)	Output head only (< 0.5%)
Memory Footprint	High (full optimizer states)	Very High (full backward graph)	Minimal (activations frozen)
Training Speed	Very Slow	Moderate	Very Fast (few epochs)
Data Requirement	Massive dataset required	Modest downstream dataset	Small downstream dataset
Convergence Risk	High risk of divergence/overfit	Stable, high accuracy	Fast, competitive accuracy
Downstream Example	Training ResNet from zero	Fine-tuning BERT end-to-end	Frozen BERT + classification head

Table 1.3: Systematic comparison of downstream adaptation strategies across computer vision and natural language processing, as presented during the laboratory lecture.

Intuition & analogy: From parameter freezing to PEFT and LoRA

Notice the profound connection to Large Language Models: freezing the backbone and training a lightweight linear adapter on top (51,300 parameters vs. 11.2M) achieves nearly identical accuracy in a fraction of the wall-clock time while completely eliminating gradient computations across deep residual blocks.
This exact mathematical philosophy underpins **Parameter-Efficient Fine-Tuning (PEFT)** and **Low-Rank Adaptation (LoRA)** in modern LLMs: instead of fine-tuning all 7B or 70B parameters of a foundation model, engineers freeze the transformer weights and update only low-rank adapter matrices, drastically lowering VRAM thresholds and democratizing model customization.

1.8 Exercise 3 in-depth: analytical derivation and autograd DAG trace

The final exercise of the laboratory (Cells 80–84) analyzes automatic differentiation on a composite mathematical function, reconciling analytical calculus with PyTorch's autograd engine.

1.8.1 Mathematical problem definition

Given input $x = 1.0$, target $y = 3.0$, and parameters $\theta_1 = 2.0$, $\theta_2 = 3.0$, we define the predicted value $\hat{y}$ and quadratic loss $\mathcal{L}$:

$$\hat{y} = \theta_1 \theta_2 x, \quad \mathcal{L}(\theta_1, \theta_2) = (\theta_1 \theta_2 x - y)^2. \tag{1.17}$$

1.8.2 Analytical derivation of partial derivatives

We decompose the forward pass into elementary atomic operations:

$$a = \theta_1 \theta_2, \tag{1.18}$$
$$b = a \cdot x, \tag{1.19}$$
$$c = b - y, \tag{1.20}$$
$$\mathcal{L} = c^2. \tag{1.21}$$

Applying the multivariate chain rule in reverse:

$$\frac{\partial \mathcal{L}}{\partial c} = 2c = 2(b - y) = 2(\theta_1 \theta_2 x - y), \tag{1.22}$$
$$\frac{\partial \mathcal{L}}{\partial b} = \frac{\partial \mathcal{L}}{\partial c} \frac{\partial c}{\partial b} = 2c \cdot 1 = 2c, \tag{1.23}$$
$$\frac{\partial \mathcal{L}}{\partial a} = \frac{\partial \mathcal{L}}{\partial b} \frac{\partial b}{\partial a} = 2c \cdot x = 2cx. \tag{1.24}$$

Now computing the parameter sensitivities:

$$\frac{\partial \mathcal{L}}{\partial \theta_1} = \frac{\partial \mathcal{L}}{\partial a} \frac{\partial a}{\partial \theta_1} = (2cx) \cdot \theta_2 = 2(\theta_1 \theta_2 x - y)\theta_2 x, \tag{1.25}$$
$$\frac{\partial \mathcal{L}}{\partial \theta_2} = \frac{\partial \mathcal{L}}{\partial a} \frac{\partial a}{\partial \theta_2} = (2cx) \cdot \theta_1 = 2(\theta_1 \theta_2 x - y)\theta_1 x. \tag{1.26}$$

1.8.3 Numerical evaluation

Substituting the problem's values $\theta_1 = 2.0, \theta_2 = 3.0, x = 1.0, y = 3.0$:

$$a = 2.0 \times 3.0 = 6.0, \tag{1.27}$$
$$b = 6.0 \times 1.0 = 6.0, \tag{1.28}$$
$$c = 6.0 - 3.0 = 3.0, \tag{1.29}$$
$$\mathcal{L} = 3.0^2 = \mathbf{9.0}. \tag{1.30}$$

Evaluating the gradients:

$$\frac{\partial \mathcal{L}}{\partial \theta_1} = 2(3.0) \times 3.0 \times 1.0 = \mathbf{18.0}, \tag{1.31}$$
$$\frac{\partial \mathcal{L}}{\partial \theta_2} = 2(3.0) \times 2.0 \times 1.0 = \mathbf{12.0}. \tag{1.32}$$

1.8.4 PyTorch implementation and computational graph visualization

```python
import torch
from torchviz import make_dot

# Define parameter and input tensors with gradient tracking
theta1 = torch.tensor(2.0, requires_grad=True)
theta2 = torch.tensor(3.0, requires_grad=True)
x = torch.tensor(1.0, requires_grad=True)
y = torch.tensor(3.0, requires_grad=True)

# Forward pass: compute scalar loss
loss = (theta1 * theta2 * x - y)**2

# Visualize computational graph
dot = make_dot(loss, params={'theta1': theta1, 'theta2': theta2, 'x': x, 'y': y})

# Execute reverse-mode automatic differentiation
loss.backward()

print("Computed Loss:", loss.item())
print("dL/d(theta1):", theta1.grad.item())
print("dL/d(theta2):", theta2.grad.item())
```

Listing 1.16: Verified solution for Exercise 3 autograd calculation and visualization (Cells 83–84).

```
Computed Loss: 9.0
dL/d(theta1): 18.0
dL/d(theta2): 12.0
```

PyTorch's automated backward traversal produces exactly 18.0 and 12.0, perfectly confirming our analytic derivation. As highlighted during the laboratory lecture, invoking `torchviz.make_dot` exposes PyTorch's internal C++ autograd backward DAG:

- **Accumulation Nodes**: Leaf variables declared with `requires_grad=True` (θ_1, θ_2, x, y) terminate at `AccumulateGrad` nodes that accumulate incoming adjoint gradients into each tensor's `.grad` buffer.
- **Backward Operators**: Non-leaf operation nodes (such as `MulBackward0` or `SubBackward0`) represent the local multivariate derivative closures executed when `loss.backward()` is triggered.

Figure 1.5 provides a vector visualization of this exact Directed Acyclic Graph, contrasting forward computation with reverse-mode adjoint propagation.

1.9 Laboratory summary and engineering checklist

Table 1.4 synthesizes the core PyTorch modules, classes, and design patterns mastered in Laboratory 01.

1.9.1 Bridge to Laboratory 02

In Laboratory 01, we established the fundamental tensor mechanisms of deep learning, progressing from scalar regression to convolutional transfer learning. In **Laboratory 02**, we transition from continuous computer vision pixels to discrete natural language text. We will explore how subword tokenizers (Hugging Face `AutoTokenizer`) map text into discrete token IDs, examine the

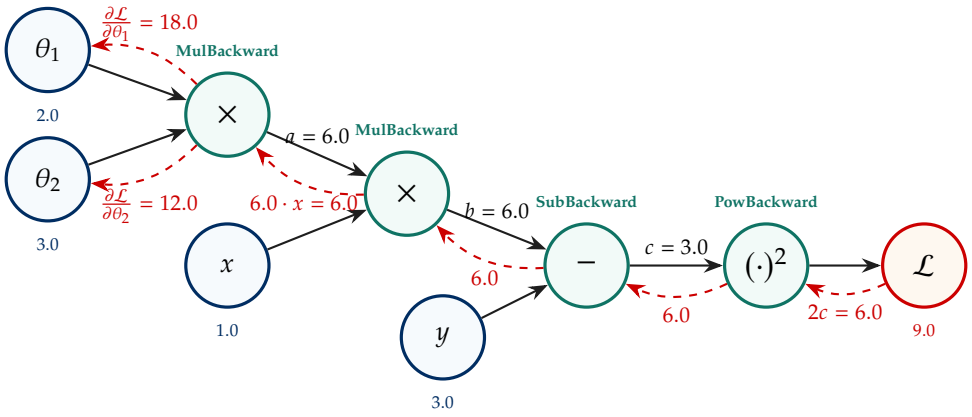


Figure 1.5: Vector rendering of the computational graph for Exercise 3 (reproducing the notebook's `computational_graph.png` and `backpropagation.png`). Black solid paths trace forward evaluation; red dashed arrows trace reverse-mode adjoint gradient propagation via the chain rule.

PyTorch API / Component	Engineering Function & Best Practice
`torch.tensor(..., requires_grad=True)`	Leaf node instantiation with autograd sensitivity tracking.
`torch.randn(shape)`	Gaussian initialization; prefer FP32/BF16 over FP64 for throughput.
`nn.Module`	Object-oriented base class for all neural layers and architectures.
`nn.Linear(in_features, out_features)`	Affine matrix transformation $\mathbf{Wx} + \mathbf{b}$ with automatic parameter registration.
`TensorDataset(X, y)`	Packs input and label tensors into a unified iterable indexing structure.
`DataLoader(dataset, batch_size, shuffle)`	Provides minibatching, shuffling, and multi-threaded parallel worker loading.
`optimizer.zero_grad()`	Resets parameter gradients to zero before backpropagation to prevent accumulation.
`loss.backward()`	Executes reverse-mode DAG topological traversal and accumulates partial derivatives.
`optimizer.step()`	Updates model parameters along the negative gradient vector.
`model.eval()`	Sets evaluation mode; freezes stochastic layers (e.g., dropout and batchnorm).
`torch.no_grad()`	Context manager deactivating the autograd graph engine to conserve VRAM.
`param.requires_grad = False`	Parameter freezing; isolates feature backbones during transfer learning.

Table 1.4: Summary of foundational PyTorch APIs mastered in Laboratory 01.

encoder-decoder architecture of **T5**, and construct the **Scaled Dot-Product Attention mechanism** ($QK^\top/\sqrt{d_k}$) from scratch.

Part II

Large language models for software engineering

Bibliography

[1] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.